Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

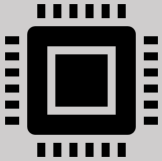
División de Ciencias de la ingeniería

Ingeniería

Organización de Lenguajes y Compiladores 2 Sección “A”

Ing. Mario Moises Ramirez Tobar y Yennifer de Leon

Segundo Semestre 2026



Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Manual técnico Proyecto #2



Descripción:

En el siguiente documento se da a conocer los detalles a tener en cuenta sobre la presente práctica. Sobre todo si se desea ampliar las funcionalidades de la aplicación se deben de conocer a detalle cómo es que funciona la gestion de archivos utilizada durante la practica.

Disclaimer: Todo el código se encuentra escrito en inglés

“Bienvenido programador”

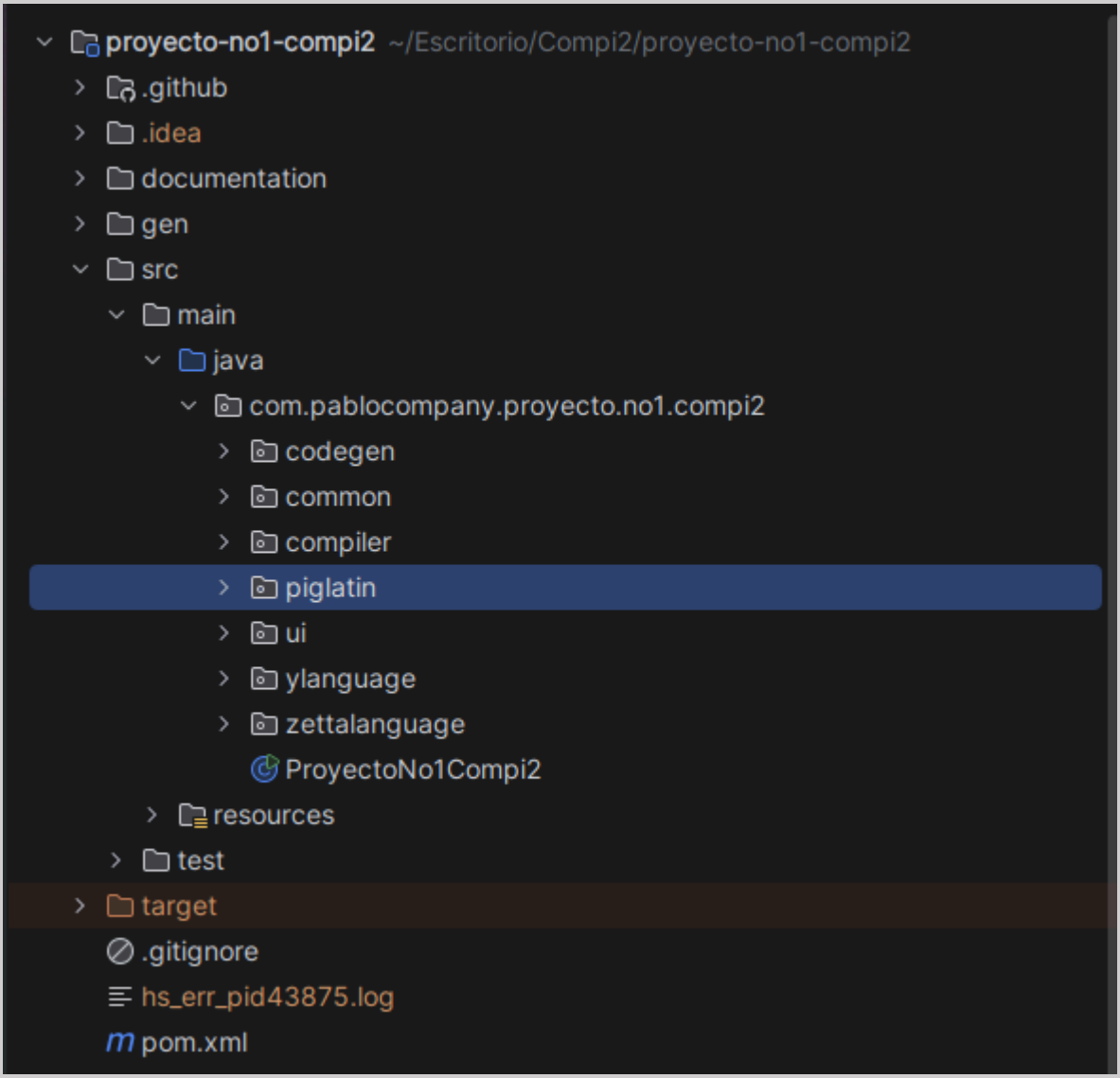
Como primer punto se requiere contar con los pasos para poder ejecutar el código del repositorio en la computadora local del programador, al igual que conocer la distribución de carpetas que se hizo. No sin antes mencionar todas las metodologías, tecnologías y patrones de diseño utilizados.

Tecnologías utilizadas para la realización del proyecto:

- **Java:** Utilizada como lenguaje principal sobre el que se montó la aplicación. Esta cuenta con su respectivo código dividido y organizado donde se puede interactuar tanto con el backend como en el frontend.
- **ANTLR:** El principal gestor de analisis lexico y sintactico
- **Java Swing:** Es el principal motor gráfico con el que cuenta la aplicación y todos los renderizados que hace internamente están montados en esta librería.
- **Maven:** Principal gestor de dependencias del proyecto.
- **Github:** Repositorio remoto de la aplicación.

DIRECTORIO DE ARCHIVOS

Debido a que el proyecto base se realizó con cierta distribución de archivos. Así que se le recomienda al nuevo programador que mantenga el estándar que se maneja en el nombramiento de variables.



Detalles importantes:

- **La carpeta “compiler”:** es donde se encuentra toda la lógica del intérprete y de la demás lógica de negocio que maneja la aplicación.
- **La carpeta “application”:** es donde se aloja toda la capa de comunicación y puertos principales de gestión de la aplicación
- **La carpeta: “infrastructure”:** Maneja toda la lógica de negocio perteneciente a la herramienta utilizada y todo lo relacionado con la tecnología que conlleva.
- **La carpeta: “domain”:** Maneja todas las clases que generan la representación física de la aplicación y no depende de ninguna otra capa.

Paquete “ui”

Este paquete es el que gestiona por completo las vistas de la ui usando componentes personalizados escritos en java y sobreescritura de las propias librerías de swing**. Es importante que el nuevo desarrollador mantenga las prácticas de fragmentación en componentes. nunca generar clases enormes para tareas, siempre respetar los principios SOLID.**

IMPORTANTE:

Si se desean crear más paquetes es válido. Solo que si se debe de respetar la distribución base de los paquetes ya mencionados para seguir manteniendo el formato de orden. Sobre todo si hay que agregar documentación se le pide al programador que lo adjunte dentro del paquete llamado **documentación.**

TECNOLOGÍAS UTILIZADAS

Arquitectura de software:

Arquitectura Hexagonal

Patrones de diseño:

1. Patrón experto
2. Patrón visitor
3. Patrón Observer
4. Patrón Listener
5. Patrón Facade
6. Patrón interprete (a mínima escala sin sustituir al visitor, solo lo implementa)
7. Patrón Strategy
8. **Patrón abstract factory**

Paradigmas utilizados:

- Programación orientado a objetos
- Programación funcional

Principios SOLID

Otras tecnologías utilizadas:

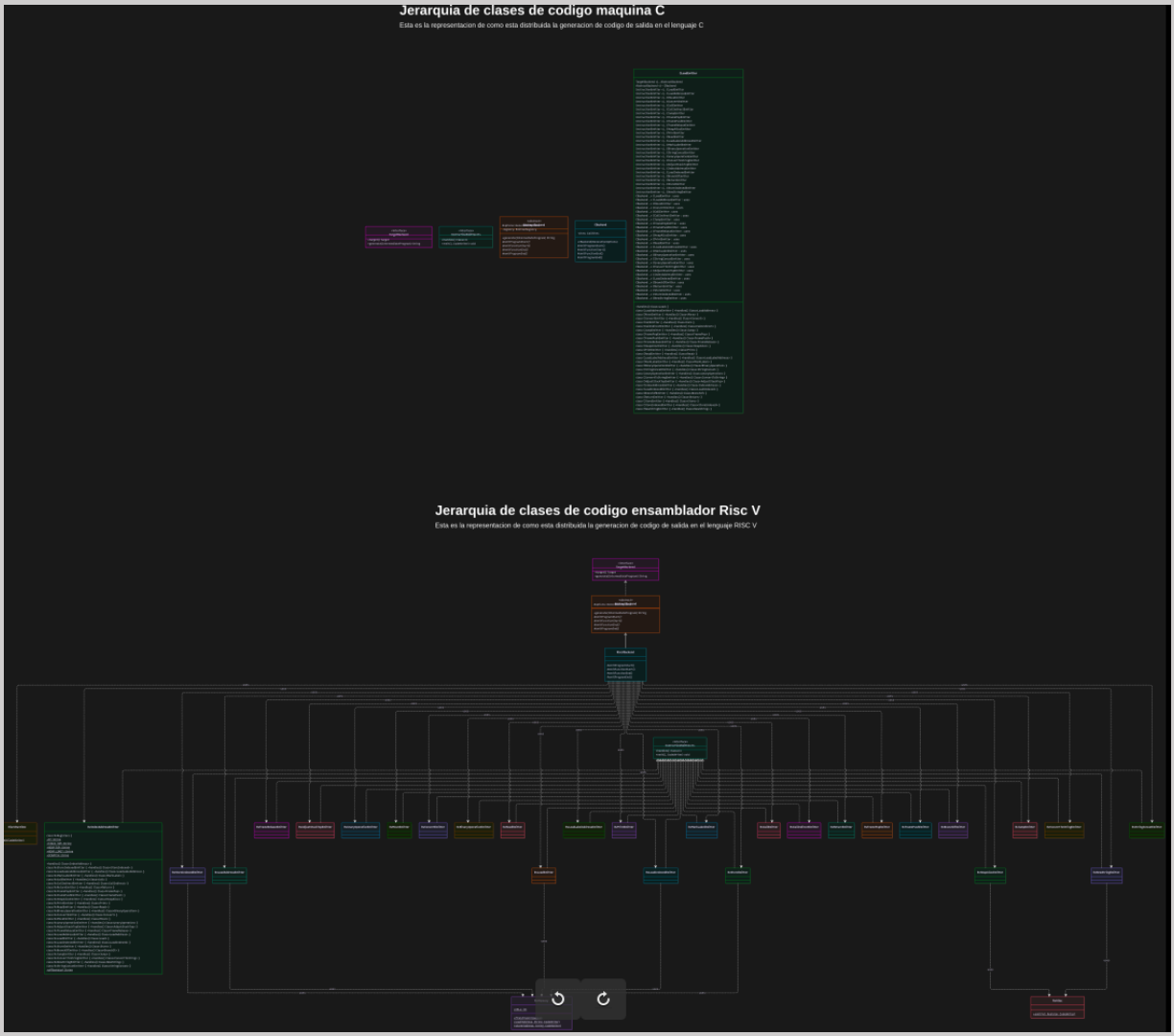
- Modelo MVC

Debido a que la aplicación fue desarrollada en el entorno de Maven, IntelliJ y Netbeans, se recomienda tener instalado:

- IDE: ya sea netbeans o IntelliJ con la extensión ANTLR
- Tener instalada la versión 21 de Java
- **Tener instalada la dependencia de ANTLR4**

DIAGRAMA DE CLASES DE CÓDIGO INTERMEDIO

Aca se deja la implementación base:

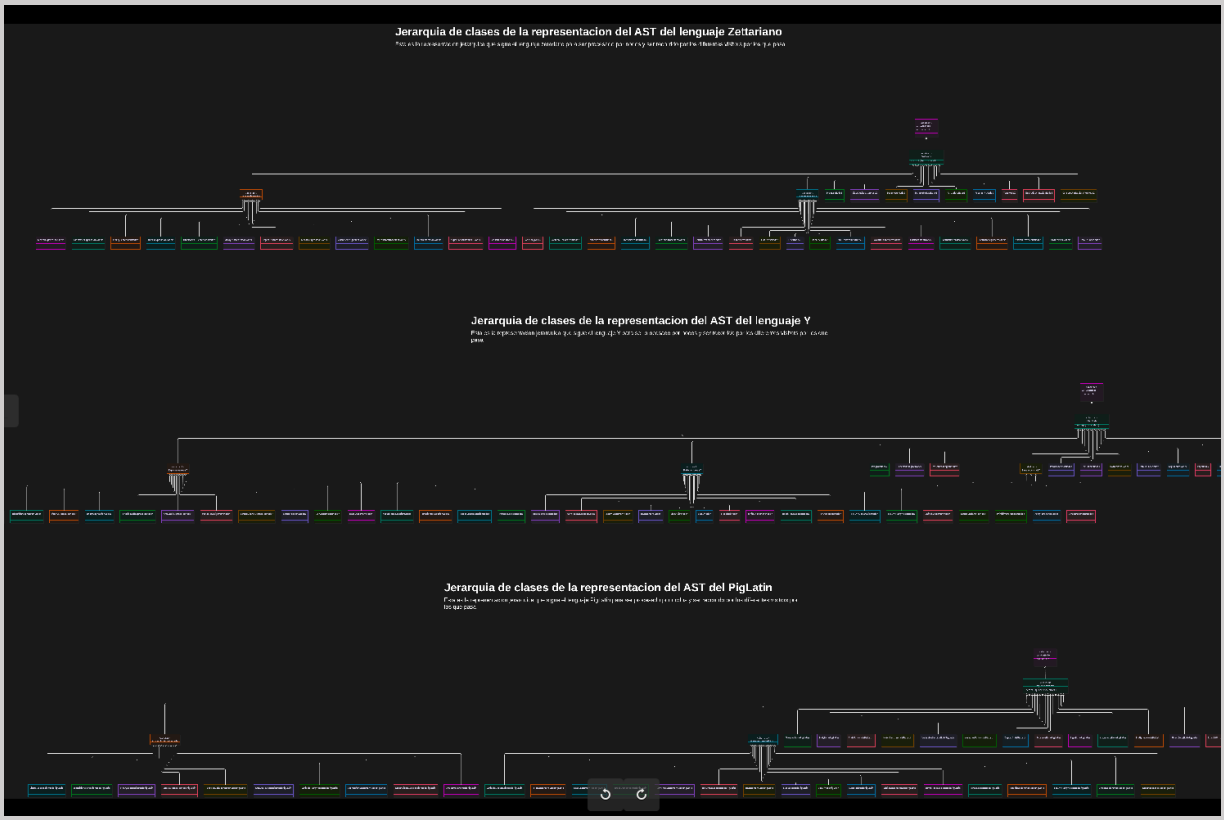


Puedes verlo más a detalle en el link:

https://drive.google.com/file/d/15-rJTkrNZRPVgMYuJAO0-zP8SdrIGAWb/view?usp=sharing

DIAGRAMA DE CLASES DEL AST

Aca se deja la implementación base:



Puedes verlo más a detalle en el link:

https://drive.google.com/file/d/18xeEZ4ikrcM9mWbimk2MjWYZcejCYeAR/view?usp=sharing

SÍMBOLOS Y PALABRAS RESERVADAS

En el análisis léxico se tomaron las siguientes palabras como reservadas:

Se lleva a cabo un análisis léxico gracias a ANTLR que permite generar un **autómata finito determinista** en base a los principios del método de Thompson hasta poder llegar a un autómata con todos los estados que permita validar los estados de aceptación. Por lo tanto el método consiste en:

- Declaración de Expresiones regulares
- Generación de **AFN por el método de Thompson**
- Formulación de **AFD en base al algoritmo de cerradura Lambda**
- **Minimización de estados**

Nombre del archivo:

PigLatinLexer.g4

Palabras reservadas del lenguaje:

```
lexer grammar PigLatinLexer;



//====================================

// LEXICAL PART OF THE DOCUMENT

//====================================




//IGNORED CHARACTERS

COMMENT

   : '//' ~[\r\n]*

     -> channel(HIDDEN)

   ;
```

```
BLOCK_COMMENT
   : '##'+ .*? '##'+
     -> channel(HIDDEN)
   ;

WS: [ \t\r\n]+
   -> channel(HIDDEN)
   ;



//SECTION OF CODE BLOCK
VARIABILES:'VARIABILES';
MAIOR:'MAIOR';
FINIS_SEPARATOR: 'FINIS';

//SECTION OF FUNCTION ACTIONS
PRINT: '>>';
READ: '<<';

//KEYWORDS FOR OBJECTS
NULL: 'null';



//SECTION OF VARIABLE TYPES
NUMERUS: 'numerus';
```

```
TEXTUM: 'textum';

DECIMALIS: 'decimalis';

LITTERA: 'littera';

BOOLEAN: 'bool';


//BOOLEAN VALUES

VERUM: 'verum';

FALSUS: 'falsus';


//IMPORTS SECTION

IMPORT: 'import';


//POINTER SECTION

NOVUS: 'novus';


//SECTION OF SPECIAL CHARACTERS

ESTO: 'esto';

SERIES: 'series';

STRUCTURE: 'structura';

FINIS: 'finis';


DUM: 'dum';

FACERE: 'facere';

PER: 'per';

SI: 'si';

ALITER: 'aliter';
```

```
ACTIO: 'actio';

RATIO: 'ratio';

REDDERE: 'reddere';


//BREAK ACTIONS

PERGE: 'perge';

INTERRUMPE: 'interrumpe';


//SECTION OF SPECIAL OPERATORS OR PUNCTUATION

EQUAL: '=';

COMMA: ',';

DOT_COMMA: ';';

TWO_POINTS: ':';

DOT: '.';


//SECTION OF THE GROUPING SYMBOLS

INIT_BRACE: '{';

FINAL_BRACE: '}';


INIT_BRACKET: '[';

FINAL_BRACKET: ']';


INIT_PARENT: '(';

FINAL_PARENT: ')';
```

```
//SECTION TO THE ABREVIATION OPERATORS


BY_ONE_ADD: '+=';

BY_ONE_MINUS: '-=';

BY_ONE_MULTIPLICATION: '*=';

BY_ONE_DIVISION: '/=';

BY_ONE_PERCENT: '%=';


//SECTION TO THE ABREVIATION OPERATORS


ABREV_PLUS: '++';

ABREV_MINUS: '--';



//SECTION OF ARITHMETIC OPERATORS


PLUS: '+';

MINUS: '-';

MULTIPLICATION: '*';

DIVIDE: '/';

PERCENT: '%';



//SECTION OF RELATIONAL OPERATORS


EQUALS: '==';
```

```
GREATER_EQUALS: '>=';

LESS_EQUALS: '<=';

DIFERENCE: '!=';

LESS: '<';

GREATER: '>';


//SECTION OF LOGIC OPERATORS


AND: '&&';

OR: '||';

NOT: 'non';


//SECTION OF VALUES

ID: [a-zA-Z_] [a-zA-Z0-9_]* ;


INT: [0-9]+ ;


DECIMAL: [0-9]+ '.' [0-9]+ ;


STRING: '"' ( '\\' . | ~["\\\r\n] )* '"' ;


CHAR: '\'' ( '\\' . | ~['\\\r\n] ) '\'' ;

ERROR_TOKEN

   : .

   ;
```

En el análisis sintáctico se definieron las siguientes estructuras de terminales y no terminales:

Cabe aclarar que aquí simplemente se están agregando los **terminales** y los **no terminales**. En este caso el uso de ANTLR genera el reconocimiento automatizado de errores y el solito genera su recuperación de errores sintácticos. Basta con solo definir la gramática especificando terminales y usando los tokens reconocidos por el lexer y el solito te genera sus respectivos walkers para la fase de parseo.

Nombre del archivo:

PigLatinParser.g4

```
parser grammar PigLatinParser;


options {
   tokenVocab=PigLatinLexer;
}


program
   : body? EOF
   ;

/*===*****=========*****===== BODY SECTION ===*****==========*****===*/

body
   : import_list* variable_section? maior_section FINIS_SEPARATOR DOT_COMMA
   ;

/*===*****=========*****===== BODY SECTION ===*****==========*****===*/
```

```
/*------ IMPORT LIST PRODUCTION ------*/
import_list
   : IMPORT accessor       #ImportLists
   ;

/*------ IMPORT ACCESSOR PRODUCTION ------*/
accessor
   : ID (DOT ID)*          #AccessorInstruction
   ;

/*===*****===== MAIOR SECTION =====*****===*/

/*===*****========*****===== MAIOR SECTION ===*****==========*****===*/
maior_section
   : MAIOR GREATER code_body # MaiorSection
   ;

code_body
   : statement*        #CodeBodyStatement
   ;

statement
   : block_code                          # BlockCode
   | console_actions                     # ConsoleActions
   | loop_control                        # LoopControlAction
```

```
   | return_control                        # ReturnControlAction
   | abbreviated_operation                 # LocalAbbreviatedOperation
   | assignment                            # LocalAssignment
   | call_statement DOT_COMMA              # StatementObjectPropertyCalling
   | declaration                           # LocalDeclaration
   ;

/*------ CALL AS STATEMENT (solo llamadas, nunca acceso plano) ------*/
call_statement
    : object_values DOT ID INIT_PARENT arguments_list? FINAL_PARENT    #
ChainedCallStatement
    | ID INIT_PARENT arguments_list? FINAL_PARENT                      #
DirectCallStatement
   ;



/*===*****===== COMMON CODE SECTION =====*****===*/

block_code
   : if_statement              # CodeBlockIf
   | while_statement           # CodeBlockWhile
   | do_while_statement        # CodeBlockDoWhile
   | for_statement             # CodeBlockFor
   ;

/*------ IF STATEMENT PRODUCTION ------*/
```

```
if_statement
   : SI INIT_PARENT expression FINAL_PARENT INIT_BRACE code_body FINAL_BRACE
else_if_list? else_statement FINIS DOT_COMMA # IfStatement
   ;

/*------ ELSE IF LIST STATEMENT PRODUCTION ------*/
else_if_list
   : else_if_clause+       #ElseIfList
   ;

else_if_clause
     :  ALITER  INIT_PARENT  expression  FINAL_PARENT  INIT_BRACE  code_body
FINAL_BRACE      # ElseIfClause
   ;

else_statement
   : (ALITER INIT_BRACE code_body FINAL_BRACE)?       # ElseStatement
   ;

/*------ LOOPS ------*/

while_statement
      :  DUM  INIT_PARENT  expression  FINAL_PARENT  INIT_BRACE  code_body
FINAL_BRACE FINIS DOT_COMMA # WhileStatement
   ;
```

```
do_while_statement

    : FACERE INIT_BRACE code_body FINAL_BRACE DUM INIT_PARENT expression
FINAL_PARENT DOT_COMMA # DoWhileStatement

   ;


for_statement

    : PER INIT_PARENT for_init DOT_COMMA expression DOT_COMMA for_update
FINAL_PARENT INIT_BRACE code_body FINAL_BRACE  # ForStatement

   ;


for_init

   : ESTO ID TWO_POINTS variable_type expression # ForInitVarDecl

   | ID EQUAL expression                         # ForInitAssign

   ;


for_update

   : nest_variable ABREV_PLUS         #ForUpdateIncrement

   | nest_variable ABREV_MINUS        #ForUpdateDecrement

   | ABREV_PLUS nest_variable         #ForUpdatePrefixIncrement

   | ABREV_MINUS nest_variable        #ForUpdatePrefixDecrement

   | nest_variable EQUAL expression   #ForUpdateAssign

   | compound_assignment               # ComPoundAssingment

   ;


/*------ RETURN STATEMENT ------*/
```

```
return_control
   : REDDERE expression? DOT_COMMA         # ReturnStatement
   ;

/*------ LOOP CONTROL STATEMENTS ------*/

loop_control
   : PERGE DOT_COMMA               # LoopContinue
   | INTERRUMPE DOT_COMMA          # LoopBreak
   ;

/*===*****===== CONSOLE ACTIONS SECTION =====*****===*/

console_actions
   : nest_variable READ                    # ReadVariableInput
   | READ                                  # ReadInput
   | PRINT print_function DOT_COMMA        # PrintAction
   ;

print_function
   : print_function PRINT expression # PrintMultipleExpr
   | expression                      # PrintSingleExpr
   ;

/*===*****===== VARIABILES SECTION =====*****===*/
```

```
variable_section
   : VARIABILES GREATER variabiles_body    #VariablesSection
   ;

/*------ DECLARATE VARIABILES SECTION ------*/

variabiles_body
   : declaration+          #VariabilesBody
   ;

/*------ DECLARATIONS PRODUCTIONS SECTION------*/

declaration
   : variable_declaration          # VariableInstance
   | array_declaration             # NormalArrayInstance
   | assignment                    # VariableAssignment
   | abbreviated_operation         # GlobalAbbreviatedOperation
   | compound_assignment DOT_COMMA # CompoundAssignment
   ;



/*-----VARIABLE PRODUCTIONS-----*/

variable_declaration
```

```
    : ESTO ID TWO_POINTS variable_type expression DOT_COMMA                #
VariableDeclaration
    | ESTO ID TWO_POINTS object_initializer TWO_POINTS ID DOT_COMMA        #
ObjectDeclaration
    ;


/*------ OBJECT INITIALIZER (INSTANCIA DIRECTA O NULL, TIPO EXPLICITO AL
FINAL) ------*/
object_initializer
    : NOVUS ID INIT_PARENT arguments_list? FINAL_PARENT   # ObjectInitNew
    | NULL                                                # ObjectInitNull
    ;


/*-----ASSIGNMENT PRODUCTIONS-----*/


assignment
    : nest_variable EQUAL expression DOT_COMMA # StructAssignment
    ;


/*------ COMPOUND ASSIGNMENT PRODUCTIONS ------*/
compound_assignment
     :  nest_variable  BY_ONE_ADD  expression                          #
CompoundAddAssignment
     |  nest_variable  BY_ONE_MINUS  expression                        #
CompoundSubAssignment
       |  nest_variable  BY_ONE_MULTIPLICATION  expression             #
CompoundMulAssignment
```

```
      |  nest_variable  BY_ONE_DIVISION  expression                          #
CompoundDivAssignment
      |  nest_variable  BY_ONE_PERCENT  expression                           #
CompoundModAssignment
   ;


/*-----ARRAY PRODUCTIONS-----*/


array_declaration
      :  SERIES  ID  (INIT_BRACKET  expression  FINAL_BRACKET)+  TWO_POINTS
variable_type array_initialization? DOT_COMMA      # NormalArrayDeclaration
   ;



array_initialization
   : INIT_BRACE values_array_list FINAL_BRACE      # ArrayInitWithValues
   ;


values_array_list
   : expression (COMMA expression)*
   ;



/*-----STRUCT PROPERTY ACCESS -----*/


/*--------****--- ASSIGNMENT / OPERATION TARGET ---****--------*/
nest_variable
```

```
   : object_values   # NestedVariable
   ;


/*--------****--- OBJECT AND ARRAY ACCESS CHAINS ---****--------*/
object_values
   : object_values DOT ID                                          #
ObjectPropertyChain
   | object_values DOT ID INIT_PARENT arguments_list? FINAL_PARENT      #
ObjectMethodChain
   | object_values INIT_BRACKET expression FINAL_BRACKET               #
ObjectArrayAccessChain
   | ID INIT_PARENT arguments_list? FINAL_PARENT                       #
BaseFunctionCall
   | ID                                                            #
BaseIdentifier
   ;


arguments_list
   : expression (COMMA expression)*           #ArgumentsList
   ;


/*--******-------****--- OPERATION SECTION ---****-------******--*/


expression
                    :       INIT_PARENT       expression       FINAL_PARENT
# ExpressionParents
                     |       op=(NOT       |       MINUS)       expression
# ExpressionUnary
```

```
      | expression op=(MULTIPLICATION | DIVIDE | PERCENT) expression
# ExpressionMultDiv
              | expression op=(PLUS | MINUS) expression
# ExpressionAddSub
      | expression op=(LESS | GREATER | LESS_EQUALS | GREATER_EQUALS)
expression      # ExpressionRelational
            | expression op=(EQUALS | DIFERENCE) expression
# ExpressionEquality
                        | expression AND expression
# ExpressionAnd
                        | expression OR expression
# ExpressionOr
                                        | normal_values
# ExpressionValue
   ;

/*--------****--- VALUES AND TYPES SECTION ---****--------*/

variable_type
   : TEXTUM    # TypeText
   | NUMERUS   # TypeInt
   | DECIMALIS # TypeDecimal
   | LITTERA   # TypeChar
   | BOOLEAN   # TypeBoolean
   | ID        # TypeCustomId
   ;

/*--------****--- PRINCIPAL VALUES DATA ---****--------*/
```

```
normal_values
   : STRING                                          # ValString
   | CHAR                                            # ValChar
   | DECIMAL                                         # ValDecimal
   | INT                                             # ValInt
   | boolean_values                                  # ValBool
   | NULL                                            # ValNull
   | object_values                                   # ValStructNestValue
     |  array_initialization                                              #
ValArrayInitialLiteral
   | NOVUS ID INIT_PARENT arguments_list? FINAL_PARENT # ValNewInstance
   ;


boolean_values
   : VERUM     # BoolTrue
   | FALSUS    # BoolFalse
   ;
/*-----ABBREVIATED OPERATIONS-----*/
abbreviated_operation
   : nest_variable ABREV_PLUS DOT_COMMA        # IncOperation
   | nest_variable ABREV_MINUS DOT_COMMA       # DecOperation
   | ABREV_PLUS nest_variable  DOT_COMMA       # IncPrevOperation
   | ABREV_MINUS nest_variable  DOT_COMMA      # DecPrevOperation
   ;
```

YLexer.g4

```
lexer grammar YLexer;


//VIRTUAL TOKENS
tokens {
    INDENT,
    DEDENT
}



// IGNORED CHARACTERS
COMMENT
    : '//' ~[\r\n]*
      -> channel(HIDDEN)
    ;

BLOCK_COMMENT
    : '/*' .*? '*/'
      -> channel(HIDDEN)
    ;

// El NEWLINE solo consume the first enter.
// This is the principal flag to init the stack analysis.
NEWLINE
    : ( '\r'? '\n' | '\r' )
    ;
```

```
WS
    : [ \t]+
      -> channel(HIDDEN)
    ;



//SECTION OF CODE BLOCK
STRUCTURES_REGION:'%estructuras';
FUNCTION_REGION: '%funciones';



//SECTION OF SPECIAL OPERATORS
LAMBDA: '->';



//SECTION OF FUNCTION ACTIONS
PRINT: 'imprimir';
READ: 'leer';



//SECTION OF KEYWORDS

//--- IF-ELSE
DEFINE: 'definir';
```

```
STRUCT: 'estructura';

IF: 'si';

ELSE IF: 'sino';

ELSE: 'contrario';

SO: 'entonces';


//--- SWITCH-CASE

SWITCH:'elegir';

CASE:'caso';

BREAK:'romper';

ALWAYS:'siempre';


//--- FOR-LOOP

FOR: 'para';


//--- WHILE-LOOP

WHILE: 'mientras';


//--- DO-WHILE-LOOP

DO: 'hacer';


//--- BREAK FLOW ACTIONS

RETURN: 'retornar';

CONTINUE: 'continuar';
```

```
//SECTION OF VARIABLE TYPES
INTEGER: 'entero';
STRING: 'cadena';
FLOAT: 'flotante';
CHARACTER: 'caracter';
BOOLEAN: 'bool';



//BOOLEAN VALUES
TRUE: 'verdadero';
FALSE: 'falso';



//SECTION OF SPECIAL OPERATORS OR PUNCTUATION
EQUAL: '=';
COMMA: ',';
DOT_COMMA: ';';
TWO_POINTS: ':';
DOT: '.';

//SECTION OF THE GROUPING SYMBOLS
INIT_BRACE: '{';
FINAL_BRACE: '}';

INIT_BRACKET: '[';
```

```
FINAL_BRACKET: ']';


INIT_PARENT: '(';

FINAL_PARENT: ')';



//SECTION TO THE ABREVIATION OPERATORS


BY_ONE_ADD: '+=';

BY_ONE_MINUS: '-=';

BY_ONE_MULTIPLICATION: '*=';

BY_ONE_DIVISION: '/=';

BY_ONE_PERCENT: '%=';


//SECTION TO THE ABREVIATION OPERATORS


ABREV_PLUS: '++';

ABREV_MINUS: '--';



//SECTION OF ARITHMETIC OPERATORS


PLUS: '+';

MINUS: '-';

MULTIPLICATION: '*';

DIVIDE: '/';
```

```
PERCENT: '%';



//SECTION OF RELATIONAL OPERATORS


EQUALS: '==';
GREATER_EQUALS: '>=';
LESS_EQUALS: '<=';
DIFERENCE: '!=';
LESS: '<';
GREATER: '>';


//SECTION OF LOGIC OPERATORS


AND: '&&';
OR: '||';
NOT: '!';



//SECTION OF VALUES
ID: [a-zA-Z_] [a-zA-Z0-9_]* ;


INT: [0-9]+ ;


DECIMAL: [0-9]+ '.' [0-9]+ ;
```

```
TEXT: '"' ( '\\' . | ~["\\\r\n] )* '"' ;


CHAR: '\'' ( '\\' . | ~['\\\r\n] ) '\'' ;



ERROR_TOKEN
   : .
   ;
```

YParser.g4

```
parser grammar YParser;


options {
   tokenVocab=YLexer;
}


/*Initial production*/
program
   : skip_lines? structures_region? functions_region skip_lines? EOF
   ;



/*---*******---- STRUCTURES PRODUCTIONS ----*******---*/
structures_region
   : STRUCTURES_REGION NEWLINE+ struct_declaration*
#StructRegionDeclaration
```

```
    ;


/*** STRUCT DECLARATION ****/

struct_declaration

    : STRUCT ID TWO_POINTS NEWLINE skip_lines? INDENT struct_field* DEDENT
skip_lines?      #StructDeclaration

    ;


/*** STRUCT FIELDS ****/

struct_field

    : type ID NEWLINE+
#StructNormalProperty

    | type ID (INIT_BRACKET expression FINAL_BRACKET)+ NEWLINE+
#StructArrayProperty

    ;


/*---********---- FUNCTIONS PRODUCTIONS ----********---*/


functions_region

    : FUNCTION_REGION NEWLINE+ function_declaration*        #FunctionSection

    ;


/*** FUNCION DECLARATION FIELDS ****/


function_declaration
```

```
   : DEFINE ID INIT_PARENT parameter_list? FINAL_PARENT (LAMBDA type)?
TWO_POINTS NEWLINE skip_lines? INDENT statement* DEDENT skip_lines?
#FunctionDeclaration

   ;


/*** PARAMETER FIELDS ****/

parameter_list

   : parameter (COMMA parameter)*      #ParameterList

   ;


/*** FUNCTION PARAMETER FIELDS ****/


parameter

   : type ID
#PrimitiveParameter

   | (INIT_BRACKET FINAL_BRACKET)+ type ID
#ArrayParameter

   | INIT_BRACE FINAL_BRACE (INIT_BRACKET FINAL_BRACKET)* ID ID
#StructParameter

   ;




/*---*********---- INSTRUCTION AND TYPES PRODUCTIONS ----*********---*/


/*** VARIABLE STATEMENTS ****/

statement

   : block_statement                          #StatementBlock
```

```
    | console_actions                          #StatementConsoleAction
    | loop_control                             #StatementLoopControl
    | abbreviated_operation NEWLINE+
#StatementAbbreviatedOperation
    | compound_assignment NEWLINE+             #StatementCompoundAssignment
    | struct_declaration                       #StatementStructDeclaration
    | variable_declaration NEWLINE+            #VariableDeclarationStatement
    | assignment NEWLINE+                      #AssignmentVariableStatement
    | function_call NEWLINE+                   #StatementFunctionCall
    | RETURN expression? NEWLINE+              #ReturnStatement
    ;

/*---********---- CONTROL FLOW PRODUCTIONS ----********---*/

block_statement
    : if_statement                    #BlockIf
    | switch_statement                #BlockSwitch
    | while_statement                 #BlockWhile
    | do_while_statement              #BlockDoWhile
    | for_statement                   #BlockFor
    ;

/*** IF PRODUCTIONS ****/
if_statement
    : IF INIT_PARENT expression FINAL_PARENT SO NEWLINE skip_lines? INDENT
statement* DEDENT else_if_list? else_block?   #IfStatement
```

```
    ;


/*** ELSE-IF LIST PRODUCTION ****/
else_if_list
    : else_if_clause+       #ElseIfList
    ;


/*** ELSE-IF CLAUSULE PRODUCTION ****/
else_if_clause
    : ELSE_IF INIT_PARENT expression FINAL_PARENT SO NEWLINE skip_lines?
INDENT statement* DEDENT   #ElseIfClause
    ;


/*** ELSE PRODUCTION ****/
else_block
    : ELSE NEWLINE skip_lines? INDENT statement* DEDENT     #ElseBlock
    ;


/*** SWITCH PRODUCTIONS ****/
switch_statement
    : SWITCH INIT_PARENT expression FINAL_PARENT TWO_POINTS NEWLINE
skip_lines? INDENT switch_case* always_case? DEDENT   #SwitchStatement
    ;


/*** CASE PRODUCTION ****/
switch_case
```

```
    : CASE expression TWO_POINTS NEWLINE skip_lines? INDENT statement* DEDENT
#SwitchCase
    ;

/*** DEFAULT PRODUCTION ****/
always_case
    : ALWAYS TWO_POINTS NEWLINE skip_lines? INDENT statement* DEDENT
#AlwaysCase
    ;

/*** LOOPS PRODUCTIONS ****/

while_statement
    : WHILE INIT_PARENT expression FINAL_PARENT DO NEWLINE skip_lines? INDENT
statement* DEDENT   #WhileStatement
    ;

/*** DO WHILE PRODUCTION ****/

do_while_statement
    : DO TWO_POINTS NEWLINE skip_lines? INDENT statement* DEDENT WHILE
INIT_PARENT expression FINAL_PARENT NEWLINE+   #DoWhileStatement
    ;

/*** FOR PRODUCTIONS STATEMENTS ****/
for_statement
```

```
    : FOR INIT_PARENT for_init? DOT_COMMA expression? DOT_COMMA for_update?
FINAL_PARENT TWO_POINTS NEWLINE skip_lines? INDENT statement* DEDENT
#ForStatement
    ;

/*** FOR INIT STATEMENT PRODUCTION ****/
for_init
    : type ID EQUAL expression          #ForInitVarDecl
    | nest_variable EQUAL expression    #ForInitAssign
    ;

/*** FOR UPDATE STATEMENT PRODUCTION ****/
for_update
    : nest_variable ABREV_PLUS         # ForUpdateIncrement
    | nest_variable ABREV_MINUS        # ForUpdateDecrement
    | ABREV_PLUS nest_variable         # ForUpdatePrefixIncrement
    | ABREV_MINUS nest_variable        # ForUpdatePrefixDecrement
    | nest_variable EQUAL expression   # ForUpdateAssign
    | compound_assignment              # ComPoundAssingment
    ;

/*** BREAK FLOW PRODUCTIONS ****/
loop_control
    : CONTINUE NEWLINE+   #LoopContinue
    | BREAK NEWLINE+      #LoopBreak
    ;
```

```
/*---*********---- CONSOLE ACTIONS PRODUCTIONS ----*********---*/

console_actions

   : PRINT INIT_PARENT expression FINAL_PARENT NEWLINE+   #PrintAction

   | read_call NEWLINE+                                    #ReadAction

   ;


/*** READ CALL PRODUCTION ****/

read_call

   : READ INIT_PARENT FINAL_PARENT   #ReadCall

   ;


/*------ INCREMENT / DECREMENT PRODUCTIONS (PREFIX AND SUFIX) ------*/

abbreviated_operation

   : nest_variable ABREV_PLUS     # IncSufixOperation

   | nest_variable ABREV_MINUS    # DecSufixOperation

   | ABREV_PLUS nest_variable     # IncPrefixOperation

   | ABREV_MINUS nest_variable    # DecPrefixOperation

   ;


/*---*******---- COMPOUND ASSIGNMENT PRODUCTIONS ----*********---*/

/*------ COMPOUND ASSIGNMENT PRODUCTIONS ------*/

compound_assignment

   : nest_variable BY_ONE_ADD expression                #
CompoundAddAssignment
```

```
   | nest_variable BY_ONE_MINUS expression                #
CompoundSubAssignment
   | nest_variable BY_ONE_MULTIPLICATION expression      #
CompoundMulAssignment
   | nest_variable BY_ONE_DIVISION expression            #
CompoundDivAssignment
   | nest_variable BY_ONE_PERCENT expression             #
CompoundModAssignment
   ;


/*** ASSIGNMENT DECLARATIONS ****/
assignment
   : nest_variable EQUAL expression         #AssingmentStatement
   ;




/*--------****--- ASSIGNMENT / OPERATION TARGET ---****--------*/
nest_variable
   : object_values   # NestedVariable
   ;


/*--------****--- OBJECT AND ARRAY ACCESS CHAINS ---****--------*/
object_values
   : object_values DOT ID                                       #
ObjectPropertyChain
   | object_values INIT_BRACKET expression FINAL_BRACKET        #
ObjectArrayAccessChain
```

```
   | function_call
#BaseFunctionCall
   | ID                                                                  #
BaseIdentifier
   ;


function_call:
   ID INIT_PARENT arguments_list? FINAL_PARENT                          #
FunctionCall
   ;
/*--------****--- ARGUMENTS FOR FUNCTIONS LIST---****--------*/


arguments_list
   : expression (COMMA expression)*            #ArgumentsList
   ;


/*--------****--- ARRAY VALUES ---****--------*/
array_literal
   : INIT_BRACE array_values? FINAL_BRACE   # ArrayLiteralValue
   ;


/*--------****--- ARRAY VALUES ---****--------*/


array_values
   : expression (COMMA expression)*   # ArrayValuesList
   ;
```

```
/*** VARIABLE DECLARATIONS MULTI-DIMENSIONAL ARRAYS ALLOWED ****/


variable_declaration
   : type ID
#NotDefiniedVariable
   | type ID EQUAL expression
#DefiniedVariable
   | type ID (INIT_BRACKET expression FINAL_BRACKET)+
#NotDefiniedArrayVariable
   | type ID (INIT_BRACKET expression FINAL_BRACKET)+ EQUAL expression
#DefiniedArrayVariable
   ;



/*** TYPES PRODUCTIONS ****/
type
   : INTEGER          #IntValue
   | STRING           #StringValue
   | FLOAT            #FloatValue
   | CHARACTER        #CharValue
   | BOOLEAN          #BooleanValue
   | ID               #CustomTypeValue
   ;



/*** EXPRESSION PRODUCTIONS ****/
```

```
expression
   : INIT_PARENT expression FINAL_PARENT
# ExpressionParents
   | op=(NOT | MINUS) expression
# ExpressionUnary
   | expression op=(MULTIPLICATION | DIVIDE | PERCENT) expression
# ExpressionMultDiv
   | expression op=(PLUS | MINUS) expression
# ExpressionAddSub
   | expression op=(LESS | GREATER | LESS_EQUALS | GREATER_EQUALS)
expression      # ExpressionRelational
   | expression op=(EQUALS | DIFERENCE) expression
# ExpressionEquality
   | expression AND expression
# ExpressionAnd
   | expression OR expression
# ExpressionOr
   | normal_values
# ExpressionValue
   ;


/*--------****--- PRINCIPAL VALUES DATA ---****--------*/

normal_values
   : TEXT                                    # ValueString
   | CHAR                                    # ValueChar
   | DECIMAL                                 # ValueDecimal
   | INT                                     # ValueInt
   | boolean_values                # ValBool
   | object_values                 # ValObjectAccess
```

```
   | array_literal                  # ValArrayLiteral
   | read_call                      # ValReadCall
   ;


/*** BOOLEAN VALUES ****/


boolean_values
   : TRUE     # BoolTrue
   | FALSE    # BoolFalse
   ;



/*** NEW LINE PRODUCTION ****/
skip_lines
   : NEWLINE+
   ;
```

ZLexer.g4

```
lexer grammar ZLexer;


// IGNORED CHARACTERS
COMMENT
   : '//' ~[\r\n]*
     -> channel(HIDDEN)
   ;

BLOCK_COMMENT
   : '/*' .*? '*/'
     -> channel(HIDDEN)
   ;

WS: [ \t\r\n]+
   -> channel(HIDDEN)
   ;



//SPECIAL POINTER OPERATORS
NEW:'new';
NULL: 'null';

//ACCESS MODIFIERS
PUBLIC: 'public';
PROTECTED: 'protected';
```

```
PRIVATE: 'private';


//OBJECT KEYWORDS

EXTENDS: 'extends';

OVERRIDE: '@Override';

THIS: 'this';

SUPER: 'super';


//SECTION OF FUNCTION ACTIONS


//---Print action:

READ: 'readln';

PRINTLN: 'println';

PRINT: 'print';



//SECTION OF KEYWORDS


//--- IF-ELSE

IF: 'if';

ELSE: 'else';


//--- SWITCH-CASE

SWITCH:'switch';

CASE:'case';

BREAK:'break';
```

```
DEFAULT:'default';


//--- FOR-LOOP
FOR: 'for';


//--- WHILE-LOOP
WHILE: 'while';


//--- DO-WHILE-LOOP
DO: 'do';


//--- BREAK FLOW ACTIONS
RETURN: 'return';
CONTINUE: 'continue';


//--- CLASSES
CLASS: 'class';


//SECTION OF VARIABLE TYPES
INTEGER: 'int';
DOUBLE: 'double';
CHARACTER: 'char';
BOOLEAN: 'boolean';
VOID: 'void';
```

```
//BOOLEAN VALUES

TRUE: 'true';

FALSE: 'false';



//SECTION OF SPECIAL OPERATORS OR PUNCTUATION


EQUAL: '=';

COMMA: ',';

DOT_COMMA: ';';

TWO_POINTS: ':';

QUESTION: '?';

DOT: '.';



//SECTION OF THE GROUPING SYMBOLS


INIT_BRACE: '{';

FINAL_BRACE: '}';


INIT_BRACKET: '[';

FINAL_BRACKET: ']';


INIT_PARENT: '(';

FINAL_PARENT: ')';
```

```
//SECTION TO THE ABREVIATION OPERATORS

BY_ONE_ADD: '+=';
BY_ONE_MINUS: '-=';
BY_ONE_MULTIPLICATION: '*=';
BY_ONE_DIVISION: '/=';
BY_ONE_PERCENT: '%=';



//SECTION TO THE ABREVIATION OPERATORS

ABREV_PLUS: '++';
ABREV_MINUS: '--';



//SECTION OF ARITHMETIC OPERATORS

PLUS: '+';
MINUS: '-';
MULTIPLICATION: '*';
DIVIDE: '/';
PERCENT: '%';



//SECTION OF RELATIONAL OPERATORS
```

```
EQUALS: '==';
GREATER_EQUALS: '>=';
LESS_EQUALS: '<=';
DIFERENCE: '!=';
LESS: '<';
GREATER: '>';

//SECTION OF LOGIC OPERATORS

AND: '&&';
OR: '||';
NOT: '!';



//SECTION OF VALUES
ID: [a-zA-Z_] [a-zA-Z0-9_]* ;

INT: [0-9]+ ;

DECIMAL: [0-9]+ '.' [0-9]+ ;

TEXT: '"' ( '\\' . | ~["\\\r\n] )* '"' ;

CHAR: '\'' ( '\\' . | ~['\\\r\n] ) '\'' ;
```

```
ERROR_TOKEN
   : .
   ;
```

ZParser.g4

```
parser grammar ZParser;


options {
   tokenVocab=ZLexer;
}


/*===*****========*****===== PROGRAM ROOT SECTION
===*****==========*****===*/
program
   : class_declaration EOF
   ;


/*===*****========*****===== CLASS SECTION ===*****==========*****===*/


class_declaration
   : access_modifier CLASS name=ID (EXTENDS parent=ID)? INIT_BRACE
class_member* FINAL_BRACE   # ClassDeclaration
   ;


/*------ CLASS MEMBER PRODUCTIONS (CONSTRUCTOR/ATTRIBUTES) ------*/
```

```
class_member
   : field_declaration            # ClassFieldMember
   | constructor_declaration      # ClassConstructorMember
   | method_declaration           # ClassMethodMember
   ;


/*------ FIELD PRODUCTION (ENCAPSULATION: MODIFIER REQUIRED) ------*/
field_declaration
   : access_modifier? type (INIT_BRACKET FINAL_BRACKET)* ID (EQUAL
expression)? DOT_COMMA   # FieldDeclaration
   ;



/*------ CONSTRUCTOR PRODUCTION ------*/
constructor_declaration
   : access_modifier? ID INIT_PARENT parameter_list? FINAL_PARENT
     INIT_BRACE super_call? statement* FINAL_BRACE   #
ConstructorDeclaration
   ;


/*------ SUPER CONSTRUCTOR CALL ------*/
super_call
   : SUPER INIT_PARENT arguments_list? FINAL_PARENT DOT_COMMA   #
SuperConstructorCall
   ;


/*------ METHOD PRODUCTION (OVERRIDE OPTIONAL) ------*/
```

```
method_declaration
   : OVERRIDE? access_modifier? type (INIT_BRACKET FINAL_BRACKET)* ID
INIT_PARENT parameter_list? FINAL_PARENT INIT_BRACE statement* FINAL_BRACE
# MethodDeclaration
   ;


/*------ PARAMETER LIST PRODUCTIONS ------*/
parameter_list
   : parameter (COMMA parameter)*   # ParameterList
   ;


/*------ PARAMETER PRODUCTION ------*/
parameter
   : type (INIT_BRACKET FINAL_BRACKET)* ID   # ParameterStatement
   ;


/*===*****===== STATEMENT SECTION =====*****===*/
/*===*****========*****===== STATEMENTS (METHOD/CONSTRUCTOR BODY)
===*****==========*****===*/


statement
   : block_statement                  # StatementBlock
   | console_actions                  # StatementConsoleAction
   | loop_control                     # StatementLoopControl
   | return_statement                 # StatementReturn
   | abbreviated_operation            # StatementAbbreviatedOperation
   | compound_assignment DOT_COMMA    # StatementCompoundAssignment
```

```
   | nested_variables_usage            # StatementNestedAssignment
   | variable_declaration              # StatementVariableDeclaration
   | call_statement DOT_COMMA          # StatementObjectPropertyCalling
   ;


/*------ CALL AS STATEMENT------*/
call_statement
   : SUPER DOT ID INIT_PARENT arguments_list? FINAL_PARENT           #
SuperCallStatement
   | object_values DOT ID INIT_PARENT arguments_list? FINAL_PARENT   #
ChainedCallStatement
   | ID INIT_PARENT arguments_list? FINAL_PARENT                     #
DirectCallStatement
   ;


/*------ RETURN STATEMENT ------*/
return_statement
   : RETURN expression? DOT_COMMA          # ReturnStatement
   ;


/*------ LOOP CONTROL STATEMENTS ------*/
loop_control
   : CONTINUE DOT_COMMA        # LoopContinue
   | BREAK DOT_COMMA           # LoopBreak
   ;


/*------ OUTPUT ACTIONS ------*/
```

```
console_actions
   : PRINTLN INIT_PARENT expression FINAL_PARENT DOT_COMMA         #
PrintlnAction
   | PRINT INIT_PARENT expression FINAL_PARENT DOT_COMMA           #
PrintAction
   | read_call DOT_COMMA                                           #
ReadAction
   ;

/*------ INPUT ACTIONS ------*/
read_call
   : READ INIT_PARENT FINAL_PARENT   # ReadCall
   ;

/*==*****===== COMMON CODE SECTION =====*****==*/

block_statement
   : if_statement              # BlockIf
   | switch_statement          # BlockSwitch
   | while_statement           # BlockWhile
   | do_while_statement        # BlockDoWhile
   | for_statement             # BlockFor
   ;

/*------ STATEMENT BODY PRODUCTION ------*/
statement_body
   : INIT_BRACE statement* FINAL_BRACE   # BracedStatementBody
```

```
   | statement                              # SingleStatementBody
   ;


/*------ IF STATEMENT PRODUCTION ------*/
if_statement
   : IF INIT_PARENT expression FINAL_PARENT statement_body else_if_list?
else_block?   # IfStatement
   ;


/*------ ELSE IF LIST STATEMENT PRODUCTION ------*/
else_if_list
   : else_if_clause+ # ElseIfList
   ;


/*------ ELSE IF STATEMENT PRODUCTION ------*/
else_if_clause
   : ELSE IF INIT_PARENT expression FINAL_PARENT statement_body   #
ElseIfClause
   ;


/*------ ELSE STATEMENT PRODUCTION ------*/


else_block
   : ELSE statement_body   # ElseBlock
   ;
```

```
/*------ SWITCH STATEMENT PRODUCTION ------*/

switch_statement

   : SWITCH INIT_PARENT expression FINAL_PARENT INIT_BRACE switch_case*
default_case? FINAL_BRACE   # SwitchStatement

   ;


/*------ CASE INSTRUCTION PRODUCTION ------*/

switch_case

   : CASE expression TWO_POINTS statement*   # SwitchCase

   ;


/*------ DEFAULT INSTRUCTION PRODUCTION ------*/

default_case

   : DEFAULT TWO_POINTS statement*   # DefaultCase

   ;


/*------ LOOPS ------*/


while_statement

   : WHILE INIT_PARENT expression FINAL_PARENT statement_body   #
WhileStatement

   ;


/*------ DO WHILE STATEMENT PRODUCTION ------*/

do_while_statement

   : DO statement_body WHILE INIT_PARENT expression FINAL_PARENT DOT_COMMA
# DoWhileStatement
```

```
   ;


/*------ FOR STATEMENT PRODUCTION  (ALLOWS THE INFINITE LOOP) ------*/

for_statement

   : FOR INIT_PARENT for_init? DOT_COMMA expression? DOT_COMMA for_update?
FINAL_PARENT statement_body   # ForStatement

   ;


/*------ FOR INITIAL STATEMENT PRODUCTION (INITIAL ITERATOR VALUE) ------*/

for_init

   : type (INIT_BRACKET FINAL_BRACKET)* ID EQUAL expression   #
ForInitVarDecl

   | nest_variable EQUAL expression                                    #
ForInitAssign

   ;


/*------ FOR UPDATE STATEMENT PRODUCTION------*/

for_update

   : nest_variable ABREV_PLUS         #ForUpdateIncrement

   | nest_variable ABREV_MINUS        #ForUpdateDecrement

   | ABREV_PLUS nest_variable         #ForUpdatePrefixIncrement

   | ABREV_MINUS nest_variable        #ForUpdatePrefixDecrement

   | nest_variable EQUAL expression   #ForUpdateAssign

   | compound_assignment              # ComPoundAssingment

   ;


/*===*****===== VARIABLES AND ASSIGNMENT SECTION =====*****===*/
```

```
/*------ VARIABLE DECLARATION PRODUCTION (ALLOW MULTI-DIMENSIONAL
ARRAYS)------*/

variable_declaration

   : type (INIT_BRACKET FINAL_BRACKET)* ID (EQUAL expression)? DOT_COMMA   #
VariableDeclaration

   ;



/*------ NESTED ASSIGNMENT VALUES PRODUCTIONS------*/

nested_variables_usage

   : object_values EQUAL expression DOT_COMMA   # NestedObjectAssignment

   ;



/*------ COMPOUND ASSIGNMENT PRODUCTIONS ------*/

compound_assignment

   : nest_variable BY_ONE_ADD expression                #
CompoundAddAssignment

   | nest_variable BY_ONE_MINUS expression              #
CompoundSubAssignment

   | nest_variable BY_ONE_MULTIPLICATION expression     #
CompoundMulAssignment

   | nest_variable BY_ONE_DIVISION expression           #
CompoundDivAssignment

   | nest_variable BY_ONE_PERCENT expression            #
CompoundModAssignment

   ;
```

```
/*------ INCREMENT / DECREMENT PRODUCTIONS (PREFIX AND SUFIX) ------*/
abbreviated_operation
   : nest_variable ABREV_PLUS DOT_COMMA    # IncSufixOperation
   | nest_variable ABREV_MINUS DOT_COMMA   # DecSufixOperation
   | ABREV_PLUS nest_variable  DOT_COMMA   # IncPrefixOperation
   | ABREV_MINUS nest_variable  DOT_COMMA   # DecPrefixOperation
   ;

/*--------****--- ASSIGNMENT / OPERATION TARGET ---****--------*/
nest_variable
   : object_values   # NestedVariable
   ;

/*--------****--- OBJECT AND ARRAY ACCESS CHAINS ---****--------*/
object_values
   : object_values DOT ID                                          #
ObjectPropertyChain
   | object_values DOT ID INIT_PARENT arguments_list? FINAL_PARENT         #
ObjectMethodChain
   | object_values INIT_BRACKET expression FINAL_BRACKET                   #
ObjectArrayAccessChain
   | ID INIT_PARENT arguments_list? FINAL_PARENT                           #
BaseFunctionCall
   | SUPER DOT ID INIT_PARENT arguments_list? FINAL_PARENT                 #
BaseSuperMethodCall
   | SUPER DOT ID                                                          #
BaseSuperProperty
```

```
    | THIS                                                                    #
BaseThis
    | ID                                                                      #
BaseIdentifier
    ;



/*--------****--- ARGUMENT FUNCTION LIST ---****--------*/


arguments_list
    : expression (COMMA expression)*        # ArgumentsList
    ;


/*--------****--- OBJECT / ARRAY INSTANTIATION ---****--------*/


instantiation
    : NEW type INIT_PARENT arguments_list? FINAL_PARENT           #
ObjectInstantiation
    | NEW type (INIT_BRACKET expression FINAL_BRACKET)+           #
ArrayInstantiation
    ;


/*--------****--- ARRAY LITERAL ---****--------*/


array_literal
    : INIT_BRACE array_values? FINAL_BRACE   # ArrayLiteralValue
    ;
```

```
/*--------****--- ARRAY VALUES ---****--------*/


array_values
   : expression (COMMA expression)*   # ArrayValuesList
   ;


/*--******-------****--- OPERATION SECTION ---****-------******--*/
expression
   : INIT_PARENT expression FINAL_PARENT
# ExpressionParents
   | op=(NOT | MINUS) expression
# ExpressionUnary
   | expression op=(MULTIPLICATION | DIVIDE | PERCENT) expression
# ExpressionMultDiv
   | expression op=(PLUS | MINUS) expression
# ExpressionAddSub
   | expression op=(LESS | GREATER | LESS_EQUALS | GREATER_EQUALS)
expression     # ExpressionRelational
   | expression op=(EQUALS | DIFERENCE) expression
# ExpressionEquality
   | expression AND expression
# ExpressionAnd
   | expression OR expression
# ExpressionOr
   | <assoc=right> expression QUESTION expression TWO_POINTS expression
# ExpressionTernary
   | normal_values
# ExpressionValue
```

```
   ;


/*--------****--- VALUES AND TYPES SECTION ---****--------*/


type
   : primitive_type        # TypePrimitive
   | ID                    # TypeClassName
   ;


/*--------****--- PRIMITIVE TYPES ---****--------*/
primitive_type
   : INTEGER    # TypeInt
   | DOUBLE     # TypeDouble
   | CHARACTER  # TypeChar
   | BOOLEAN    # TypeBoolean
   | VOID       # TypeVoid
   ;


/*--------****--- PRINCIPAL VALUES DATA ---****--------*/
normal_values
   : TEXT                        # ValText
   | CHAR                        # ValChar
   | DECIMAL                     # ValDecimal
   | INT                         # ValInt
   | boolean_values              # ValBool
   | NULL                        # ValNull
```

```
   | instantiation                 # ValInstantiation
   | object_values                # ValObjectAccess
   | array_literal                # ValArrayLiteral
   | read_call                    # ValReadCall
   ;


/*--------****--- BOOLEAN VALUES ---****--------*/
boolean_values
   : TRUE     # BoolTrue
   | FALSE    # BoolFalse
   ;


/*--------****--- ACCESS MODIFIER VALUES ---****--------*/
access_modifier
   : PRIVATE   # ModifierPrivate
   | PUBLIC    # ModifierPublic
   | PROTECTED # ModifierProtected
   ;
```

Cabe destacar algo muy importante:

Al contrario de los LALR que existen para JAVA, esta herramienta utiliza ANTLR no permite generar ambigüedad, por su mecanismo que tiene de ser un LL(*permite remover esta ambigüedad asumiendo siempre que la primera producción será de prioridad ante las demás por lo que nunca generará árboles de derivación idénticos y llegará a un estado de aceptación. Tal como pasa en las expresiones, estas tienen cierta precedencia de operaciones

```
expression
   : INIT_PARENT expression FINAL_PARENT
   | op=(NOT | MINUS) expression
   | expression op=(MULTIPLICATION | DIVIDE) expression
   | expression op=(PLUS | MINUS) expression
   | expression op=(LESS | GREATER | LESS_EQUALS | GREATER_EQUALS)
expression      # ExpressionRelational
   | expression op=(EQUALS | DIFERENCE) expression
   | expression AND expression
   | expression OR expression
   | normal_values
   ;
```

IMPORTANTE:

En este caso, ANTLR tiene un mecanismo especial para las expresiones matemáticas asi que se declaran en la misma expresion y produccion con un el operador “|” y de esta forma se indicará que tienen la misma precedencia**.**

En el análisis semántico y reconocimiento de símbolos:

Se generó una implementación de tabla de símbolos con el uso de un HashMap, esto permite almacenar el valor de la variable bajo una llave única que era el identificador respectivo de la variable.

Se utilizó el mismo concepto del manejo de ámbitos, ya que como tal el único ámbito que existe es el global, se hizo uso de una implementación de pila para poder reconocer cada ámbito en su propio bloque. Lo que permite generar el reconocimiento de identificadores y el shadowing de identificadores sin generar conflictos.

Un símbolo se compone con lo siguientes atributos:

```
public class Symbol {


   private String name;
   private SymbolKind kind;
   private String type;
   private String filePath;
   private String fileName;
   private int line;
   private int column;
   private boolean isArray;
   private ParameterKind parameterKind;
   private int dimensions;
   //Null dynamic dimensions
   private List<Integer> dimensionSizes;


   // For callables: signature support (overload)
   private List<String> parameterTypes;
```

```
private String qualifiedName;

//Struct attributes
private List<Symbol> members;

private String returnType;

AccessModifierZ accessModifier;

String declaringClass;

private int returnDimensions;

boolean overriding;

String parentName;

@EqualsAndHashCode.Exclude
@ToString.Exclude
Symbol parentSymbol;

private boolean inheritanceLinked;
/**
 * Non-null when the hierarchy is invalid
 */
```

```
    private String inheritanceError;


    /**
     * Unique id of the class
     */
    private int classId = -1;
    /**
     * Label of the class virtual table
     */
    private String vtableLabel;
    /**
     * Virtual table
     */
    @EqualsAndHashCode.Exclude
    @ToString.Exclude
    private List<Symbol> vtable = new ArrayList<>();
    /**
     * Every attribute of the object in memory order: inherited ones FIRST
     */
    @EqualsAndHashCode.Exclude
    @ToString.Exclude
    private List<Symbol> fieldSlots = new ArrayList<>();


    /**
     * Label of the emitted function
     */
```

```
    private String codeLabel;

    /**
     * true if it goes through the vtable
     */
    private boolean virtual;

    /**
     * Slot in the vtable of its class
     */
    private int vtableIndex = -1;

    /**
     * Method of the superclass that this one overrides
     */
    @EqualsAndHashCode.Exclude
    @ToString.Exclude
    private Symbol overriddenMethod;


    // ---------- ATTRIBUTE ----------

    /**
     * 0-based position among the object's fields -1 if not a field.
     */
    private int fieldIndex = -1;
```

TABLA DE COMPATIBILIDAD DE TIPOS

Esta práctica se inspira completamente en cómo es que funciona C++ o C# ya que estos permiten el uso de expresiones en conjunto con valores booleanos y es fuertemente tipado. Cosa que para este proyecto era muy importante mantener un lenguaje fuertemente tipado pero con todo tipo de expresiones que pudieran surgir siempre bajo las restricciones que se imponen sobre la inferencia de tipos.

A continuación se deja el comportamiento de inferencia de tipos que realiza el lenguaje:

Tipo izquierdo	Tipo derecho	Operación	Tipo inferido
TEXTO	ENTERO	SUMA	TEXTO
TEXTO	CHAR	SUMA	TEXTO
TEXTO	DECIMAL	SUMA	TEXTO
TEXTO	BOOLEANO	SUMA	TEXTO
TEXTO	TEXTO	SUMA	TEXTO
DECIMAL	ENTERO	SUMA	DECIMAL
DECIMAL	ENTERO	RESTA	DECIMAL
DECIMAL	ENTERO	MULTIPLICACIÓN	DECIMAL
DECIMAL	ENTERO	DIVISIÓN	DECIMAL
ENTERO	CHAR	SUMA	ENTERO
ENTERO	CHAR	RESTA	ENTERO
ENTERO	CHAR	MULTIPLICACIÓN	ENTERO
ENTERO	CHAR	DIVISIÓN	ENTERO
ENTERO	BOOLEANO	SUMA	ENTERO
ENTERO	BOOLEANO	RESTA	ENTERO

ENTERO	BOOLEANO	MULTIPLICACIÓN	ENTERO
ENTERO	BOOLEANO	DIVISIÓN	ENTERO
CHAR	N/A	N/A	CHAR
BOOLEANO	N/A	N/A	BOOLEANO

Importante:

Cabe destacar que no se detallaron las demás expresiones ya que estas causan error y no tienen ningún valor como tal. Por lo tanto solo esas son las expresiones válidas que se permiten en el compilador.